

Week 4 · 실습 워크북: 값을 밖으로 빼고, 블록 하나로 두 개를 만듭니다

참고

이 문서는 실습 파트(45분)입니다. 개념은 같은 `lecture/` 폴더의 `개념워크북` 을 먼저 보세요. 여기서는 번호만 가리킵니다. [실습] 은 직접 치고, [확인] 은 눈으로 대조하고, [함정] 은 미리 피하고, [관찰 🕒] 은 빈칸을 채웁니다. (심화 ⭐) 는 먼저 끝낸 사람만 합니다.

작업 폴더는 `practice/` 입니다. `solution/` 은 막혔을 때 답을 보는 곳입니다. 위에서 아래로 그대로 따라가면 끝납니다.

오늘 만들 것

리소스 9개입니다. week3과 종류는 같고 서브넷과 라우트 테이블 연결만 2개씩으로 늘어납니다. 그 둘을 블록 하나로 만드는 것이 오늘의 실습입니다.

폴더	만드는 것	개수	state 는 어디에
<code>practice/count-demo/</code>	<code>terraform_data</code> 6개. AWS 자격증명이 필요 없습니다	6	이 폴더의 로컬 파일
<code>practice/</code>	VPC + subnet 2 + IGW + route table + 연결 2 + SG + EC2	9	week3 의 S3 버킷 (<code>week04/app/...</code>)

비용·보안 경고

비용. `practice/` 를 apply 하면 시간당 \$0.0190이 붙습니다. destroy 를 잊고 한 달 두면 약 \$13.87입니다. Block C는 선택이 아닙니다. `for_each` 를 EC2 · NAT Gateway · Elastic IP 에 걸지 마세요. 요금이 항목 수만큼 곱해집니다. count-demo 는 \$0 입니다.

올리면 안 되는 값. 본인 공인 IP · AWS 계정번호 12자리 · 액세스 키 · `terraform.tfvars` · `terraform.tfstate` · `.pem` 파일. 마스킹 명령은 C-4에 있습니다.

체크포인트

오늘도 폴더가 둘입니다. `practice/` 와 `practice/count-demo/` . count-demo 는 A-2에서만 쓰고 C-2에서 정리합니다. 이 문서의 코드블록 첫 줄에 붙은 `# pwd == ...` 주석은 그 블록의 명령을 다 친 뒤에 있어야 하는 위치입니다. 블록 안에 `cd` 가 있으면 시작 위치는 그 위 블록의 값입니다. 매 블록 끝에 `pwd` 로 대조하세요.

파일	오늘 손대나
<code>versions.tf</code> · <code>providers.tf</code> · <code>compute.tf</code>	읽기만
<code>backend.tf</code>	TODO ① week3 버킷 이름을 손으로
<code>variables.tf</code>	TODO ③③③
<code>locals.tf</code>	TODO ②②
<code>network.tf</code>	TODO ②②
<code>outputs.tf</code>	TODO ②②
<code>example.tfvars</code>	<code>terraform.tfvars</code> 로 복사해 값 채우기
<code>count-demo/main.tf</code>	완성본입니다. 값 하나만 지워봅니다

`compute.tf` 는 week3 정답 코드와 거의 같고 달라진 곳은 `subnet_id` 한 줄뿐입니다. AMI 필터를 조인 이유와 IMDSv2 는 week3 개념워크북 Part 5에 있습니다.

오늘의 지도: 값이 나가고, 블록 하나가 여러 개를 만듭니다

week1의 지도는 명령의 흐름, week2는 리소스 사이의 참조, week3은 state 파일이 사는 곳이었습니. 오늘의 지도는 값이 어디에 적혀 있는지와, 블록 하나가 리소스 몇 개가 되는지입니다.

① BEFORE: 오늘 시작 시점 (week3 이 끝난 상태)

<pre>practice/network.tf resource "aws_subnet" "public" { cidr_block = "10.0.1.0/24" availability_zone = names[0] } resource "aws_route_table_association" resource "aws_instance" "web"</pre>	AWS 계정 (서울) <table><tr><td>subnet</td><td>1</td></tr><tr><td>association</td><td>1</td></tr><tr><td>EC2</td><td>1</td></tr><tr><td>VPC · IGW · RT</td><td></td></tr><tr><td>security group</td><td></td></tr></table>	subnet	1	association	1	EC2	1	VPC · IGW · RT		security group	
subnet	1										
association	1										
EC2	1										
VPC · IGW · RT											
security group											

블록 하나가 리소스 하나입니다.
 값은 리소스 블록 안에 그대로 적혀 있습니다.
 서브넷을 하나 더 만들려면 블록을 통째로 복사해야 합니다.

㉠ AFTER: 오늘 끝날 때

variables.tf	network.tf	AWS
subnets = {	resource "aws_subnet"	subnet 2
a = { cidr, az }	=> "public" {	association 2
c = { cidr, az }	for_each = var.subnets	VPC 1
}	each.key -> Name	IGW 1
primary_subnet_key = "a"	each.value -> cidr, az	route table 1
my_ip (sensitive)	}	SG 1
vpc_cidr		EC2 1
locals.tf	resource "aws_instance"	
name_prefix	=> subnet_id =	합계 9
common_tags = merge(...)	public["a"].id	

왼쪽 상자가 오늘 새로 생긴 것입니다. 가운데 상자의 블록 수는 week3 과 같습니다.
 오른쪽에서 늘어난 것은 subnet 과 association 둘뿐이고, EC2 는 그대로 1대입니다.

㉠ 인스턴스 key 지도: 주소가 번호일 때와 key 일 때

count 로 만들면 : 주소가 번호입니다

```
terraform_data.by_count[0] "alpha"
terraform_data.by_count[1] "bravo"    <- 이 원소를 지웁니다
terraform_data.by_count[2] "charlie"
```

list 에서 "bravo" 를 지운 뒤

```
terraform_data.by_count[0] "alpha"    그대로
terraform_data.by_count[1] "charlie"  -/+ 교체
terraform_data.by_count[2] 없음      - 삭제
      ^
```

"charlie" 는 값이 하나도 안 바뀌었는데 [2] 에서 [1] 로 옮겨졌습니다.
 state 는 리소스를 주소로 기억하므로 이 이동이 곧 교체입니다.

for_each 로 만들면 : 주소가 key 입니다

```
terraform_data.by_for_each["alpha"]
terraform_data.by_for_each["bravo"]    <- 이 원소를 지웁니다
terraform_data.by_for_each["charlie"]
```

set 에서 "bravo" 를 지운 뒤

```
terraform_data.by_for_each["alpha"]    그대로
terraform_data.by_for_each["charlie"]  그대로
      ^
```

지운 것만 사라집니다. 나머지 주소는 움직이지 않습니다.

A-2 의 plan 마지막 줄이 이 그림 전체입니다. 숫자를 항목별로 나눈 표는 개념워크북 11번에 있습니다.

스텝 ↔ 지도 대응표

스텝	지도에서 하는 일
A-2	㉠을 손으로 재현합니다
A-4 · A-5	㉠의 상자에서 값을 뽑아 ㉠의 왼쪽 상자로 옮깁니다
A-7	㉠의 locals.tf 칸을 채웁니다
B-1 · B-2	㉠의 가운데 상자. 화살표 하나가 여러 개를 만듭니다

스텝	지도에서 하는 일
B-3	㉔의 오른쪽 상자를 output 으로 꺼냅니다
B-5	㉔의 오른쪽 상자를 <code>state list</code> 로 확인합니다
B-7	㉔의 <code>for_each</code> 절반을 AWS 에서 재현합니다
C-1	㉔의 오른쪽 상자만 사라집니다

오늘 나와야 하는 숫자

이 표와 다르면 멈추고 물어보세요. 넘어가면 뒤에서 더 크게 터집니다.

지점	나와야 하는 값	다르면
A-2 count-demo <code>apply</code>	Apply complete! Resources: 6 added, 0 changed, 0 destroyed.	<code>main.tf</code> 를 고쳤는지 확인
A-2 count-demo <code>state list</code>	6줄	멈춤
A-2 "bravo" 삭제 후 <code>plan</code>	Plan: 1 to add, 0 to change, 3 to destroy.	멈춤. 오늘 실습 전체의 근거입니다
A-3 <code>init</code>	Terraform has been successfully initialized!	week3 버킷 이름 확인
A-6 validation	Error: Invalid value for variable 과 내가 적은 문장	TODO ㉔㉔ 확인
B-4 <code>plan</code>	Plan: 9 to add, 0 to change, 0 to destroy.	즉시 멈춤. 아래 CAUTION
B-5 <code>state list</code>	11줄 (리소스 9 + 데이터 소스 2)	멈춤
B-5 서브넷 주소	<code>aws_subnet.public["a"]</code> · <code>aws_subnet.public["c"]</code>	<code>[0]</code> <code>[1]</code> 이면 <code>count</code> 를 쓴 것입니다
B-6 S3 객체	<code>week04/app/terraform.tfstate</code> 1개	<code>backend.tf</code> 의 <code>key</code> 확인
C-1 <code>destroy</code>	Destroy complete! Resources: 9 destroyed.	멈춤
C-2 count-demo <code>destroy</code>	Destroy complete! Resources: 6 destroyed.	멈춤
C-3 <code>state list</code>	빈 출력	멈춤

주의

B-4의 `9 to add` 가 오늘의 안전 게이트입니다. `backend.tf` 의 `key` 를 week3의 `week03/app/...` 으로 잘못 두면 week4 스택이 지난주 `state` 를 인수하고, 그 상태로 `apply` 하면 지난주 리소스를 지우거나 교체합니다. 숫자가 9가 아니면 `apply` 하지 말고 멈추세요. 코드로 막을 수 없는 자리라 이 확인이 유일한 방어선입니다. 개념워크북 16번.

Block A. 워크스루 (18분): 먼저 이유를 보고, 값을 코드 밖으로

오늘은 `for_each` 를 쓰기 전에 왜 쓰는지부터 봅시다. A-2가 그 자리이고 AWS를 건드리지 않습니다.

[확인] A-1. 사전 점검 (2분)

지금 하는 일은 도구와 week3의 저장소가 살아 있는지 보는 것입니다. `{본인-github-id}` 자리는 셸이 채워주지 않으니 week3에서 쓴 `project_name` 을 직접 넣으세요.

```
# pwd == week4-variables-loops
terraform version          # v1.15.8 이상
aws sts get-caller-identity # 계정번호와 사용자 확인

aws s3api head-bucket --bucket "boaz26-w3-{본인-github-id}-tfstate" && echo "OK: 버킷 있음"
aws dynamodb describe-table --region ap-northeast-2 \
  --table-name "boaz26-w3-{본인-github-id}-tflock" --query 'Table.TableStatus' --output text
```

OK: 버킷 있음
ACTIVE

체크포인트

둘 중 하나라도 실패하면 여기서 멈추고 강사에게 알려주세요. week3의 `bootstrap` 스택을 다시 `apply` 해야 합니다. 그때 `project_name` 은 week3에서 쓴 값과 글자 하나까지 같아야 합니다. 버킷 이름이 거기에 걸려 있습니다.

출력된 계정번호 12자리는 화면 공유나 스크린샷에 넣지 마세요.

[실습] [관찰 🍷] A-2. count-demo: 인덱스가 밀리는 것을 먼저 봅시다 (5분)

지금 하는 일은 `count` 와 `for_each` 의 차이를 눈으로 만드는 것입니다. 이 폴더는 AWS를 건드리지 않습니다. `terraform_data` 는 Terraform 내장 리소스라 provider 도 자격증명도 필요 없고 요금도 없습니다. `init` 출력에 provider 를 내려받는 줄이 없는 것도 그 때문입니다.

```
# pwd == practice/count-demo
cd practice/count-demo
terraform init
terraform apply      # yes
```

```
Apply complete! Resources: 6 added, 0 changed, 0 destroyed.
```

주소를 확인합니다.

```
# pwd == practice/count-demo
terraform state list
```

```
terraform_data.by_count[0]
terraform_data.by_count[1]
terraform_data.by_count[2]
terraform_data.by_for_each["alpha"]
terraform_data.by_for_each["bravo"]
terraform_data.by_for_each["charlie"]
```

위 셋은 번호이고 아래 셋은 key 입니다. 이제 `main.tf` 를 열어 `names` 의 기본값에서 가운데 `"bravo"` 를 지웁니다.

```
default      = ["alpha", "charlie"]
```

apply 하지 말고 `plan` 만 하세요.

```
# pwd == practice/count-demo
terraform plan
```

```
# terraform_data.by_count[1] must be replaced
-/+ resource "terraform_data" "by_count" {
  ~ id           = "126a2864-24da-76a5-f1bb-d4f94985d892" -> (known after apply)
  ~ input        = "bravo" -> "charlie"
  ~ output       = "bravo" -> (known after apply)
  ~ triggers_replace = "bravo" -> "charlie"
}

# terraform_data.by_count[2] will be destroyed
# (because index [2] is out of range for count)

# terraform_data.by_for_each["bravo"] will be destroyed
# (because key ["bravo"] is not in for_each map)

Plan: 1 to add, 0 to change, 3 to destroy.
```

Terraform 이 이유를 직접 적어줍니다. `(because index [2] is out of range for count)` 와 `(because key ["bravo"] is not in for_each map)` 두 줄이 오늘의 결론입니다.

`by_count[1]` 옆에는 `must be replaced` 만 있고 왜 교체되는지는 안 나오므로, `~ triggers_replace = "bravo" -> "charlie"` 줄을 보고 값이 밀려 들어온 것을 읽어야 합니다.

도움

`1 to add` 가 왜 있는지 헷갈립니다. 교체 한 건은 삭제 하나와 생성 하나로 세어지기 때문입니다. 개념워크북 11번에 분해표가 있습니다.

지운 줄을 되돌려 놓으세요. C-2에서 destroy 할 때 6개가 맞아야 합니다.

```
default      = ["alpha", "bravo", "charlie"]
```

A-2 기록

- `by_count` 세 줄을 그대로 옮겨 적기: _____
- `by_for_each` 세 줄을 그대로 옮겨 적기: _____
- "bravo" 삭제 후 plan 마지막 줄: Plan: ____ to add, ____ to change, ____ to destroy.
- 교체(-/+)되는 `by_count` 의 주소와 그 주소에서 바뀌는 값: _____
- `by_for_each` 쪽에서 손대는 것은 몇 개이고 무엇인가: _____

[실습] A-3. TODO ☺: week3 버킷을 이어 쓰고 `init` (4분)

지금 하는 일은 오늘 만들 것의 state 를 week3 버킷에 두는 것입니다.

```
# pwd == practice
cd ..
cp example.tfvars terraform.tfvars
curl -4 ifconfig.me      # 내 공인 IP 확인
```

`terraform.tfvars` 의 `CHANGE-ME` 두 곳을 채웁니다. `project_name` 은 week4 용으로 새로 정하고, `my_ip` 에는 위 결과를 넣습니다.

```
region      = "ap-northeast-2"
project_name = "boaz26-w4-kdh1834"
my_ip       = "1.2.3.4"
instance_type = "t3.micro"
```

주의

`my_ip` 에 `/32` 를 붙이지 마세요. 코드가 `"${var.my_ip}/32"` 로 조립하기 때문에, 이미 붙여두면 `1.2.3.4/32/32` 가 되어 검사에 걸립니다. `terraform.tfvars` 는 커밋 금지입니다. 본인 공인 IP가 들어 있습니다.

이제 `backend.tf` 를 열어 `CHANGE-ME` 두 곳을 week3 값으로 바꿉니다. 방금 정한 `project_name` 과 다른 값입니다. 버킷은 week3에서 만든 것을 그대로 씁니다.

```
terraform {
  backend "s3" {
    bucket      = "boaz26-w3-kdh1834-tfstate"
    key         = "week04/app/terraform.tfstate"
    region      = "ap-northeast-2"
    dynamodb_table = "boaz26-w3-kdh1834-tflock"
    encrypt     = true
  }
}
```

주의

`key` 는 이미 채워져 있습니다. 건드리지 마세요. week3 리포에서 `terraform output -raw backend_config` 를 복사해 오면 `key` 가 `week03/app/terraform.tfstate` 로 나옵니다. 그 줄까지 붙여넣으면 지난주 state 를 인수합니다.

```
# pwd == practice
terraform init
```

```
Terraform has been successfully initialized!
```

[함정] `Error: Failed to get existing workspaces: S3 bucket "... does not exist.` 가 나오면 버킷 이름이 틀렸거나 week3의 bootstrap 이 없는 것입니다. A-1로 돌아가세요.

[함정] `dynamodb_table` 이 deprecated 라는 경고가 함께 뜹니다. week3에서 본 그 경고이고 정상입니다.

[실습] A-4. TODO ☺☺: `subnets` 와 `sensitive` (4분)

지금 하는 일은 서버넷 정의를 코드 밖으로 빼고, 공인 IP 를 화면에서 가리는 것입니다.

`variables.tf` 의 TODO ☺ 주석을 풀어 채우세요. 값이 두 개(cidr, az)라서 `map(string)` 으로는 담을 수 없습니다.

```
variable "subnets" {
  description = "만들 퍼블릭 서버넷 목록. key 가 for_each 의 주소가 되므로 짧고 안 바뀌는 이름을 씁니다"
```

```

type = map(object({
  cidr = string
  az   = string
}))

default = {
  a = { cidr = "10.0.1.0/24", az = "ap-northeast-2a" }
  c = { cidr = "10.0.2.0/24", az = "ap-northeast-2c" }
}

```

이어지는 `validation` 네 개는 지우지 말고 그대로 두세요. 개수 상한 · CIDR 형식 · 중복 CIDR · AZ 접두어를 각각 검사합니다. 무엇을 막는지는 개념워크북 4번과 5번에 있습니다.

TODO ☹은 한 줄입니다. `my_ip` 블록 안에 넣으세요.

```
sensitive = true
```

도움

이 한 줄 때문에 plan 출력에서 시큐리티 그룹의 `ingress` 블록 전체가 `(sensitive value)` 로 가려집니다. 스크린샷에 본인 공인 IP 가 남지 않습니다.

[실습] A-5. TODO ☹: 다른 변수를 참조하는 검사 (1분)

지금 하는 일은 EC2를 놓을 서브넷을 고르는 변수를 만드는 것입니다.

```

variable "primary_subnet_key" {
  description = "EC2 를 놓을 서브넷. subnets 맵의 key 중 하나여야 합니다"
  type        = string
  default     = "a"

  validation {
    condition     = contains(keys(var.subnets), var.primary_subnet_key)
    error_message = "primary_subnet_key 는 다음 중 하나여야 합니다: ${join(" ", keys(var.subnets))}"
  }
}

```

`condition` 이 `var.subnets` 를 참조합니다. 다른 변수를 참조하는 `validation` 은 Terraform 1.9부터 됩니다. `versions.tf` 가 `>= 1.9.0` 인 이유가 여기 있습니다.

[함정] [관찰 🕒] A-6. 일부러 틀린 값을 넣어 봅니다 (1분)

지금 하는 일은 검사가 언제 걸리는지 확인하는 것입니다.

```

# pwd == practice
terraform plan -var 'primary_subnet_key=b'

```

```

Error: Invalid value for variable

on variables.tf line 118:
118: variable "primary_subnet_key" {
    |_____
    | var.primary_subnet_key is "b"
    | var.subnets is map of object with 2 elements

primary_subnet_key 는 다음 중 하나여야 합니다: a, c

```

마지막 문장이 우리가 적은 `error_message` 이고, 이 오류는 AWS에 요청을 보내기 전에 났습니다. 검사가 없었다면 plan 계산 도중에 `Invalid index` 라는 짧은 오류만 났을 것입니다.

`locals.tf` 의 TODO ☹☹은 아직 비어 있습니다. `local.name_prefix` 를 못 찾다는 오류가 함께 보이면 A-7을 먼저 채운 뒤 이 명령을 다시 실행하세요.

`my_ip` 도 한 번 터뜨려 보세요.

```

# pwd == practice
terraform plan -var 'my_ip=1.2.3.4/32'

```

A-6 기록

- 일부러 넣은 잘못된 값: _____
- 오류 첫 줄: _____
- 내가 적은 `error_message` 가 그대로 나왔는가: _____
- 이 오류는 plan 이 시작되기 전에 났는가, apply 도중에 났는가: _____

[실습] A-7. TODO ☹☹: `locals` 와 `merge()` (1분)

지금 하는 일은 여러 곳에서 쓰는 조립식을 한 자리에 모으는 것입니다.

`locals.tf` 의 주석을 풀어 채우세요.

```
locals {
  name_prefix = var.project_name

  common_tags = merge(
    var.extra_tags,
    {
      Project   = var.project_name
      Study     = "boaz-terraform-26"
      Week      = "4"
      ManagedBy = "terraform"
      Purpose   = "workload"
    }
  )
}
```

체크포인트

순서를 뒤집지 마세요. `merge()` 는 뒤에 적은 맵이 이깁니다. `var.extra_tags` 가 앞에 있어야 `Purpose = "workload"` 를 tfvars 에서 덮어쓸 수 없습니다. 이 태그가 C-3의 잔존 점검 기준입니다. 개념워크북 8번.

이 맵은 `providers.tf` 의 `default_tags` 로 들어가서 오늘 만드는 모든 리소스에 자동으로 붙고, 리소스마다 적는 태그는 `Name` 한 줄로 줄어듭니다.

Block B. 각자 진행 (20분): 블록 하나로 여러 개 만들기

[실습] B-1. TODO ☹: 서브넷을 `for_each` 로 (4분)

지금 하는 일은 블록 하나가 서브넷 두 개를 만들게 하는 것입니다.

`network.tf` 의 TODO ☹ 주석을 풀어 채우세요.

```
resource "aws_subnet" "public" {
  for_each = var.subnets

  vpc_id          = aws_vpc.main.id
  cidr_block      = each.value.cidr
  availability_zone = each.value.az

  map_public_ip_on_launch = true

  tags = { Name = "${local.name_prefix}-public-${each.key}" }

  lifecycle {
    precondition {
      condition     = contains(data.aws_availability_zones.available.names, each.value.az)
      error_message = "subnets[${each.key}].az 가 이 계정에서 쓸 수 없는 AZ입니다: ${each.value.az}"
    }
  }
}
```

`each.key` 가 "a" · "c" 이고 `each.value` 가 `{ cidr = ..., az = ... }` 객체입니다. `lifecycle` 블록은 손으로 적은 AZ 가 이 계정에 실제로 있는지 plan 단계에서 검사합니다. 왜 필요한지는 개념워크북 15번에 있습니다.

[실습] B-2. TODO @: 연결도 for_each 로 (2분)

지금 하는 일은 서버넷마다 라우트 테이블을 붙이는 것입니다.

```
resource "aws_route_table_association" "public" {
  for_each = aws_subnet.public

  subnet_id      = each.value.id
  route_table_id = aws_route_table.public.id
}
```

여기서는 `var.subnets` 대신 `aws_subnet.public` 자체를 넣습니다. `for_each`로 만든 리소스는 그 자체가 맵이라 다시 순회할 수 있고, key는 그대로 유지되면서 `each.value`가 서버넷 객체가 됩니다.

[참정] 서버넷 ID는 apply 후에 정해지는데 괜찮습니다. `for_each`가 plan 시점에 알아야 하는 것은 key 뿐이고, 그 key는 `var.subnets`에서 이미 정해졌습니다. 개념워크북 12번.

[실습] B-3. TODO @@@: for 표현식과 sensitive output (3분)

지금 하는 일은 여러 개가 된 리소스를 맵 하나로 꺼내는 것입니다.

```
output "subnet_ids" {
  description = "for_each 로 만든 서버넷의 key 와 ID 맵"
  value       = { for k, s in aws_subnet.public : k => s.id }
}
```

TODO @은 일부러 틀려보고 채우세요. 먼저 `sensitive = true` 없이 이렇게 적습니다.

```
output "ssh_source_cidr" {
  description = "시큐리티 그룹이 22번을 열어준 대역. 본인 공인 IP 이므로 가려 둡니다"
  value       = "${var.my_ip}/32"
}
```

```
# pwd == practice
terraform plan
```

Error: Output refers to sensitive values

```
on outputs.tf line 50:
50: output "ssh_source_cidr" {
```

To reduce the risk of accidentally exporting sensitive data that was intended to be only internal, Terraform requires that any root module output containing sensitive data be explicitly marked as sensitive, to confirm your intent.

`sensitive`는 전염됩니다. A-4에서 `my_ip`에 붙인 표시가 그 값을 쓰는 output까지 따라옵니다. 이제 한 줄을 추가하세요.

```
sensitive = true
```

[확인] B-4. fmt → validate → plan (2분)

지금 하는 일은 apply 전에 세 단계로 확인하는 것입니다.

```
# pwd == practice
terraform fmt
terraform validate
terraform plan
```

Plan: 9 to add, 0 to change, 0 to destroy.

주의

9가 아니면 apply 하지 말고 멈추세요. 7이면 TODO ㉠ 중 하나가 안 풀렸고, 다른 숫자가 나오거나 삭제·교체가 섞여 있으면 backend.tf 의 key 가 week3 것일 수 있습니다. 그대로 apply 하면 지난주 리소스를 건드립니다.

plan 출력에서 시큐리티 그룹 쪽을 보세요. 공인 IP 가 가려져 있습니다.

```
+ ingress          = (sensitive value)
```

[확인] [관찰 🍌] B-5. apply 와 주소 확인 (5분)

지금 하는 일은 실제로 만들고, 주소에 key 가 붙은 것을 보는 것입니다.

```
# pwd == practice
terraform apply      # yes
terraform state list
```

state list 가 11줄입니다. 데이터 소스 두 줄이 먼저 나오는데, 읽기라서 plan 의 9 to add 에는 들어가지 않습니다.

```
data.aws_ami.al2023
data.aws_availability_zones.available
aws_instance.web
aws_internet_gateway.gw
aws_route_table.public
aws_route_table_association.public["a"]
aws_route_table_association.public["c"]
aws_security_group.web
aws_subnet.public["a"]
aws_subnet.public["c"]
aws_vpc.main
```

output 을 봅니다.

```
# pwd == practice
terraform output
```

```
subnet_ids = {
  "a" = "subnet-..."
  "c" = "subnet-..."
}
ssh_source_cidr = <sensitive>
```

주소 하나를 열어봅니다.

```
# pwd == practice
terraform state show 'aws_subnet.public["a"]'
```

주의

작은따옴표로 감싸야 합니다. 안 감싸면 셸이 대괄호와 큰따옴표를 먼저 해석해서 zsh: no matches found 가 납니다.

B-5 기록

- plan 마지막 줄: Plan: ____ to add, ____ to change, ____ to destroy.
- state list 줄 수: _____ (리소스 ____ + 데이터 소스 ____)
- aws_subnet.public 으로 시작하는 두 줄: _____
- terraform output subnet_ids 가 찍은 key 두 개: _____
- terraform output ssh_source_cidr 이 화면에 찍은 값: _____

[관찰 🍌] B-6. sensitive 가 막는 것과 못 막는 것 (2분)

지금 하는 일은 화면과 저장소를 각각 확인하는 것입니다.

```
# pwd == practice
terraform output ssh_source_cidr      # 가려집니다
```

```
terraform output -raw ssh_source_cidr # 값이 나옵니다
```

state 에는 어떻게 들어 있는지 봅시다.

```
# pwd == practice
terraform state pull | grep -c "${terraform output -raw ssh_source_cidr | cut -d/ -f1}"
```

0보다 큰 숫자가 나오면 state 안에 평문으로 들어 있다는 뜻입니다. 개념워크북 6번에서 읽은 그대로입니다.

S3 객체도 확인합니다.

```
# pwd == practice
aws s3 ls "s3://boaz26-w3-{본인-github-id}-tfstate/week04/app/"
```

B-6 기록

- S3 객체 키 전체: _____
- terraform output 과 terraform output -raw 에서 값이 각각 보였는가: _____ / _____
- state 안에 평문으로 들어 있었는가: _____
- sensitive 가 막는 것과 못 막는 것을 한 문장으로: _____

[관찰 🍌] B-7. for_each 의 안정성을 AWS 에서 확인 (2분)

지금 하는 일은 A-2에서 본 것을 실제 인프라에서 확인하는 것입니다. apply 하지 말고 plan 만 하세요.

terraform.tfvars 에 서버넷 하나를 더 적습니다.

```
subnets = {
  a = { cidr = "10.0.1.0/24", az = "ap-northeast-2a" }
  b = { cidr = "10.0.3.0/24", az = "ap-northeast-2b" }
  c = { cidr = "10.0.2.0/24", az = "ap-northeast-2c" }
}
```

```
# pwd == practice
terraform plan
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
```

새 서버넷과 그 연결 두 개만 추가로 잡혀야 합니다. 기존 ["a"] 와 ["c"] 에는 ~ 도 -/+ 도 붙지 않습니다. count 였다면 가운데에 끼어든 b 때문에 뒤가 전부 밀렸을 것입니다.

확인이 끝나면 terraform.tfvars 에 방금 적은 subnets 블록을 지워 서버넷 2개로 되돌리세요. 세 번째 서버넷을 남긴 채 apply 하면 오늘 기준인 리소스 9개에서 벗어납니다.

B-7 기록

- 세 번째 key 추가 후 plan 마지막 줄: Plan: ____ to add, ____ to change, ____ to destroy.
- 기존 ["a"] · ["c"] 에 ~ 나 -/+ 가 붙었는가: _____
- A-2의 count 쪽 결과와 비교해 한 문장: _____

[실습] B-8 (심화 ☆). -backend-config 로 값 분리

지금 하는 일은 backend 값을 코드 밖 파일로 빼는 것입니다. backend.hcl 은 .gitignore 대상입니다.

backend.tf 에는 backend "s3" {} 만 남기고, backend.hcl 에 다섯 줄을 적은 뒤 이렇게 초기화합니다.

```
# pwd == practice
terraform init -backend-config=backend.hcl -reconfigure
```

주의

-reconfigure 는 state 를 복사하지 않습니다. 같은 버킷과 같은 key 를 가리키므로 안전하지만, 오타가 나면 빈 state 를 보게 됩니다. 실행 직후 terraform state list 가 11줄인지 반드시 확인하세요.

B-9와 Block C는 이 상태에서 그대로 이어집니다. 고쳐놓은 backend.tf 는 세션 후 제출 절차의 git restore practice/ 에서 되돌립니다.

[실습] B-9 (심화 🌟). terraform console 로 표현식 시험하기

지금 하는 일은 apply 없이 식을 확인하는 것입니다.

```
# pwd == practice
terraform console
```

```
> keys(var.subnets)
> values(var.subnets)[0].cidr
> merge({ a = 1, b = 2 }, { b = 99 })
> { for k, v in var.subnets : k => v.az }
> toset(["x", "y", "x"])
```

merge 의 마지막 줄에서 b 가 99가 되는 것을 확인하세요. 개념워크북 8번의 우선순위입니다. 나가려면 exit 을 칩니다.

Block C. 정리 (7분): destroy 와 잔존 점검

비용·보안 경고

이 블록을 빼면 EC2가 계속 켜져 있습니다. 시간당 \$0.0190, 한 달이면 약 \$13.87입니다. 실습의 마지막 순서로 두지 말고 독립된 필수 블록으로 보세요.

지우는 것은 오늘 만든 것뿐입니다. week3의 S3와 DynamoDB는 7주차까지 남깁니다.

[확인] C-1. practice/ destroy (3분)

B-7에서 terraform.tfvars 에 넣은 세 번째 서버넷을 지웠는지 먼저 확인하세요.

```
# pwd == practice
terraform destroy      # yes
```

```
Destroy complete! Resources: 9 destroyed.
```

[참정] destroy 가 시큐리티 그룹이나 서버넷에서 멈추면 ENI 가 남아 있는 경우입니다. 잠시 기다린 뒤 한 번 더 실행하세요.

[실습] C-2. count-demo destroy (1분)

과금은 없지만 로컬 state 가 남아 제출물에 섞입니다.

```
# pwd == practice/count-demo
cd count-demo
terraform destroy      # yes
```

```
Destroy complete! Resources: 6 destroyed.
```

6이 아니면 A-2 끝에서 "bravo" 를 되돌리지 않은 것입니다.

[확인] [관찰 🍷] C-3. state 와 계정을 따로 확인 (2분)

state list 가 비었다는 것은 Terraform 이 관리하지 않는다는 뜻일 뿐, 계정에 아무것도 없다는 뜻이 아닙니다.

```
# pwd == week4-variables-loops
cd ../../
./scripts/check-leftover.sh
```

Purpose=workload 는 전부 0이어야 하고, week3의 S3와 DynamoDB는 남아 있어야 정상입니다.

```
[1] 오늘 지웠어야 하는 것 (Purpose=workload)
OK   EC2 인스턴스 (과금!)           0
OK   미사용 EBS 볼륨 (과금!)        0
OK   Elastic IP (과금!)             0
OK   NAT Gateway (과금!)            0
OK   workload VPC (무료·개수제한)    0
```

```
[2] 7주차까지 남겨 두는 것 (week3 에서 만든 state 저장소)
유지 state S3 버킷                boaz26-w3-kdh1834-tfstate
    ↳ week04/ 객체 1개 (state가 원격에 있으면 1개)
유지 잠금 DynamoDB 테이블        boaz26-w3-kdh1834-tflock
```

aws CLI가 없으면 콘솔에서 EC2 인스턴스 · EBS 볼륨 · Elastic IP · NAT Gateway 네 가지를 서울 리전에서 직접 확인하세요.

비용·보안 경고

stop 은 destroy 가 아닙니다. 인스턴스를 멈추면 EC2와 퍼블릭 IPv4 요금은 멈추지만 EBS 8GiB의 월 \$0.73은 계속 나갑니다.

C-3 기록

- Purpose=workload 항목들이 전부 0인가:
- week3의 S3 버킷과 DynamoDB 테이블은 남아 있는가:
- count-demo 의 state list 는 비었는가:

[실습] C-4. 제출 전 마스킹 (1분)

지금 하는 일은 증빙에서 가려야 할 값을 지우는 것입니다.

```
# pwd == practice
cd practice                # C-3 에서 리포 루트로 올라왔으므로 다시 내려갑니다
ID=본인-github-id         # 예시 값 그대로 두지 마세요
echo "$ID"                 # 바꿨는지 눈으로 확인
mkdir -p "../submissions/$ID"

terraform state list > "../submissions/$ID/state-list.txt"
```

C-1에서 destroy 를 했으므로 이 파일은 비어 있습니다. B-5의 11줄을 증빙으로 내려면 그때 화면을 따로 저장해 두세요.

계정번호 12자리와 IP 를 가립니다. macOS 와 GNU 에서 sed -i 문법이 다르므로 파일을 새로 만드는 쪽이 안전합니다.

```
# pwd == practice
sed -E \
-e 's/[0-9]{12}/<account-id>/g' \
-e 's/([0-9]{1,3}\.){3}[0-9]{1,3}/x.x.x.x/g' \
"../submissions/$ID/state-list.txt" > "../submissions/$ID/masked.txt"
mv "../submissions/$ID/masked.txt" "../submissions/$ID/state-list.txt"
```

커밋 직전에 확인합니다. 아무것도 안 나와야 정상입니다.

```
# pwd == week4-variables-loops
cd ..
git diff --cached | grep -nE '([0-9]{1,3}\.){3}[0-9]{1,3}|[0-9]{12}|AKIA[0-9A-Z]{16}'
git status --porcelain | grep -E 'terraform\.tfvars|\.tfstate|\.pem'
```

비용·보안 경고

스크린샷도 마스킹하세요. 명령으로 걸러지지 않습니다. 가려야 하는 것은 공인 IP와 계정번호 12자리입니다.

[개념] C-5. week3 저장소는 그대로 둡니다

읽고 넘어가세요. week3에서 만든 S3 버킷과 DynamoDB 테이블은 7주차까지 남기고, 5주차와 6주차도 같은 버킷에 key 만 바꿔 씁니다. 지우는 절차는 week3 실습워크북 C-5에 있고 이번 주에는 실행하지 않습니다.

세션 후: 실습 결과 제출 (브랜치 → PR)

세션 중에는 그냥 practice/ 에서 실습하면 됩니다. 끝나고 아래를 하세요.

```
# pwd == week4-variables-loops
ID=본인-github-id          # {github-id} 는 셸이 치환해주지 않습니다

git switch main && git pull
```

```
git switch -c "week4/$ID"

mkdir -p "submissions/$ID"
cp practice/*.tf "submissions/$ID/" # backend.tf 포함
cp practice/.terraform.lock.hcl "submissions/$ID/"
# C-4에서 저장한 state-list.txt 는 이미 그 폴더에 있습니다 (마스킹 확인!)
# 관찰 🕒 답안을 submissions/$ID/observations.md 로 정리

git add "submissions/$ID"
git status # tfvars / tfstate / backend.hcl 안 올라갔는지!
git commit -m "week4: $ID 실습 제출"
git push -u origin "week4/$ID"
```

넣을 것	넣지 말 것
practice/ 의 .tf 파일 (backend.tf 포함)	terraform.tfvars
example.tfvars (값은 채우지 않은 상태)	terraform.tfstate · *.backup
state-list.txt (마스킹 완료)	backend.hcl
count-demo 의 plan 출력 (A-2)	.pem 파일
observations.md (관찰 🕒 답안)	공인 IP · 계정번호 12자리

복사가 끝나면 practice/ 는 되돌려 줍니다.

```
# pwd == week4-variables-loops
git restore practice/ # 복사를 끝낸 뒤에만! 스텝레톤으로 되돌아갑니다
rm -f practice/backend.hcl # 내가 만든 파일은 git 이 모릅니다
```

그다음 GitHub에서 week4/{github-id} 에서 main 으로 PR을 올리면 강사가 리뷰 후 머지합니다.

체크포인트

제출은 submissions/{github-id}/ 본인 폴더에 하세요. practice/ 를 직접 고쳐서 올리면 머지하는 순간 다음 사람이 풀 # T0D0 가 사라집니다.

체크포인트 (Definition of Done)

- ☐ A-1: week3 버킷과 테이블이 살아 있는 것 확인
- ☐ A-2: count-demo 에서 Plan: 1 to add, 0 to change, 3 to destroy. 재현
- ☐ A-3: init 성공. key 가 week04/app/terraform.tfstate
- ☐ A-6: 일부러 틀린 값으로 validation 오류 재현
- ☐ B-4: plan 이 Plan: 9 to add, 0 to change, 0 to destroy.
- ☐ B-5: state list 11줄. 서브넷 주소가 aws_subnet.public["a"] 형태
- ☐ B-5: subnet_ids 가 key 두 개짜리 맵으로 나옴
- ☐ B-6: sensitive 가 화면은 가리고 state 는 못 가리는 것 확인
- ☐ B-7: 서브넷을 추가해도 기존 두 개가 움직이지 않는 것 확인
- ☐ C-1: destroy 완료 (9 destroyed)
- ☐ C-2: count-demo destroy 완료 (6 destroyed)
- ☐ C-3: Purpose=workload 전부 0
- ☐ C-4: 마스킹 확인
- ☐ week3의 S3와 DynamoDB는 지우지 않았습니다

오늘의 명령 요약

```
# Block A (practice/count-demo)
terraform init && terraform apply # 6 added
terraform state list # 6줄. 번호와 key 를 대조
# main.tf 의 names 에서 "bravo" 삭제
terraform plan # 1 to add, 0 to change, 3 to destroy
# "bravo" 를 되돌려 놓기
```

```

# Block A (practice)
cp example.tfvars terraform.tfvars      # project_name · my_ip
#   backend.tf 의 CHANGE-ME 두 곳을 week3 값으로 (key 는 건드리지 않기)
terraform init                          # week3 버킷의 week04/app/ 키
#   TODO ㉔~㉖ 채우기
terraform plan -var 'primary_subnet_key=b'  # validation 오류 재현

# Block B (practice)
#   TODO ㉗~㉙ 채우기
terraform fmt && terraform validate
terraform plan                          # Plan: 9 to add <- 아니면 멈춤
terraform apply                        # yes
terraform state list                   # 11줄
terraform output                      # subnet_ids 맵 · ssh_source_cidr <sensitive>
terraform state show 'aws_subnet.public["a"]'  # 작은따옴표 필수

# Block C
terraform destroy                      # 9 destroyed
cd count-demo && terraform destroy      # 6 destroyed
cd ../.. && ./scripts/check-leftover.sh # workload 전부 0

```

다음 주 예고

Week 5. 모듈화 [대면]. 오늘 변수화한 코드를 재사용 단위로 묶습니다.

오늘 본 것	Week5에서 문제가 되는 지점
<code>variables.tf</code> 하나에 변수 8개가 모였습니다	다른 폴더에서 쓰려면 복사해야 합니다. 복사본은 곧 원본과 어긋납니다
<code>subnet_ids</code> 를 맵으로 꺼냈습니다	지금은 사람이 화면으로 읽습니다. 코드가 코드에게 넘기려면 받는 쪽이 필요합니다
<code>for_each</code> 로 서브넷을 2개로 늘렸습니다	리소스 하나가 아니라 리소스 묶음을 여러 벌 만들려면 어디에 걸까요
<code>local.common_tags</code> 가 이 폴더 안에서만 보입니다	<code>locals</code> 는 폴더 경계를 넘지 않습니다
<code>backend.tf</code> 에 버킷 이름을 또 손으로 적었습니다	<code>dev</code> 와 <code>prod</code> 를 같은 코드로 만들려면 <code>key</code> 만 달라야 합니다

예습: `module` 블록, `source`, 모듈의 input 과 output

부록. 트러블슈팅

증상	원인	해결
<code>Invalid for_each argument</code>	<code>list</code> 를 그대로 넣음	<code>map</code> 을 쓰거나 <code>toset(list)</code> 로 감싸기
The "for_each" value depends on resource attributes...	<code>key</code> 에 아직 모르는 값을 씀	<code>key</code> 는 변수와 상수로만 만들기. 값 쪽에는 리소스 속성을 써도 됩니다
<code>Invalid index</code>	<code>primary_subnet_key</code> 가 <code>subnets</code> 에 없음	TODO ㉔의 <code>validation</code> 확인
Error: Invalid value for variable	<code>validation</code> 위반	<code>error_message</code> 가 요구하는 형식으로 값 수정
Error: Resource precondition failed	<code>subnets</code> 의 <code>az</code> 가 이 계정에 없음	<code>aws ec2 describe-availability-zones --region ap-northeast-</code>
Output refers to sensitive values	<code>sensitive</code> 변수를 참조하는 output 에 표시 없음	그 output 에도 <code>sensitive = true</code>
<code>zsh: no matches found</code>	주소의 대괄호를 셸이 해석	작은따옴표로 감싸기
Failed to get existing workspaces: S3 bucket ... does not exist	week3 버킷 이름 오타 또는 bootstrap 없음	A-1로 돌아가기
<code>AccessDenied</code> on <code>init</code>	IAM 권한 부족	week3 개념워크북 Part 3의 권한 목록
<code>plan</code> 이 9가 아닌 이상한 숫자	<code>backend.tf</code> 의 <code>key</code> 가 week3 것	<code>apply</code> 하지 말고 <code>key</code> 를 <code>week04/app/...</code> 으로
Error acquiring the state lock	동시 실행 또는 비정상 종료	아무도 작업 안 하는 것 확인 후 <code>terraform force-unlock <LOCK_ID></code>
Warning: Deprecated Parameter	<code>dynamodb_table</code> 사용	정상입니다. week3 개념워크북 9번

증상	원인	해결
destroy 가 SG/서브넷에서 멈춤	ENI 잔존	잠시 후 재실행

참고 문서

- [for_each](#) : B-1 · B-2
- [count](#) : A-2
- [terraform_data](#) : A-2
- [Custom Conditions](#) : A-5 · A-6 · B-1
- [for Expressions](#) : B-3
- [Sensitive data in state](#) : B-6
- [terraform console](#) : B-9

이 문서의 count-demo 출력(6 added · Plan: 1 to add, 0 to change, 3 to destroy.) 와 그 위 세 블록), app 스택의 Plan: 9 to add , state list 11줄과 정렬 순서, Error: Invalid value for variable · Output refers to sensitive values · + ingress = (sensitive value) 는 Terraform 1.15.8 · AWS provider 6.58.0 · 서울 리전에서 실제로 실행해 얻은 값입니다. B-7의 Plan: 2 to add 와 C-1의 9 destroyed 는 위 검증값에서 계산한 값이고 실제 apply 로는 확인하지 않았습니다. 비용은 개념워크북 Part 6과 같습니다.

체크포인트

끝났으면 practice/ 와 practice/count-demo/ 를 둘 다 destroy 했는지, 그리고 week3의 S3와 DynamoDB는 건드리지 않았는지 한 번 더 확인하세요. EC2가 켜져 있으면 시간당 \$0.0190이 계속 불고, week3 버킷을 지우면 5주차부터 처음부터 다시 만들어야 합니다.